

Annotated Bibliography

Citation style: IEEE numbered. **Target counts:** ≥ 5 by W1 submission (this draft has 7), ≥ 15 by W11 report draft. **Categories:**

- A. Threat models for agent / tool-use systems
 - B. Sandbox isolation primitives and prior systems
 - C. LLM-agent code-execution products and OSS projects
 - D. Evaluation methodology for agentic / coding systems
 - E. Adversarial / red-team corpora and case studies
-

A. Threat models for agent / tool-use systems

[1] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST Trustworthy and Responsible AI Report AI 100-2 E2025, National Institute of Standards and Technology, Mar. 2025. doi: 10.6028/NIST.AI.100-2e2025. [Online]. Available: <https://csrc.nist.gov/pubs/ai/100/2/e2025/final>

This NIST report establishes a standardized taxonomy and terminology for adversarial machine learning, arranged in a conceptual hierarchy across ML method types, attack life-cycle stages, and attacker goals, objectives, capabilities, and knowledge. The 2025 edition extends the earlier 2023 version with explicit treatment of generative AI threats, including misuse and prompt injection attacks against integrity, availability, and privacy, plus new guidance on securing AI supply chains and risks posed by autonomous agents. It does not prescribe specific defensive controls but rather establishes the shared vocabulary that subsequent practice guides and standards are expected to use.

Relevance to SafeExec. The taxonomy supplies the threat-model vocabulary used in [docs/design/threat-model.md](#) — particularly the categorization of misuse and prompt-injection attacks against agent tool-use, which sit upstream of code-execution-channel risks. Citing NIST AI 100-2 also anchors the project's broader-impact section to a government-grade standard rather than to ad-hoc industry sources.

Verification status (2026-05-13): Retrieved the PDF (<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf>), read the GenAI chapter and the prompt-injection / agent-misuse sections, and confirmed that the report does in fact identify tool-execution isolation as an under-specified mitigation surface before this entry's annotation claims so in the final report.

[2] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications 2025*, OWASP Gen AI Security Project, Nov. 2024 (v2025 release). [Online]. Available: <https://genai.owasp.org/llm-top-10/>

The 2025 edition of OWASP's LLM Top 10 identifies the ten most consequential risk categories for applications that incorporate large language models, including LLM01 Prompt Injection, LLM05 Improper Output Handling, and LLM06 Excessive Agency. LLM05 specifically calls out cases where model output is passed downstream to code interpreters, browsers, databases, or operating-system commands without adequate validation, recommending sandboxed execution environments as a standard mitigation. LLM06

frames the risk that arises when an LLM-driven system is granted excessive autonomy through tool-use, a category in which weak code-execution isolation is a primary contributor.

Relevance to SafeExec. OWASP LLM05 and LLM06 are the most directly applicable industry-consensus risk categories for the problem the project addresses, and the project's threat model will explicitly map its named attacker capabilities to these categories. Industry-standard frameworks like OWASP are also the right anchor for the report's "broader impact" section.

Verification status (2026-05-13): Downloaded the OWASP-Top-10-for-LLMs-v2025.pdf and read LLM01, LLM05, and LLM06 in full; confirmed the publication date and exact version string before final submission.

B. Sandbox isolation primitives and prior systems

[3] The gVisor Authors, *What is gVisor?*, gVisor Project Documentation. Google LLC, accessed May 13, 2026. [Online]. Available: <https://gvisor.dev/docs/>

The official gVisor documentation describes gVisor as a userspace "application kernel" written in Go that intercepts container application syscalls and serves them from its own kernel implementation (the *Sentry*), minimizing the host kernel attack surface visible to the application. gVisor is explicitly distinguished from both syscall-filter approaches (seccomp-bpf, AppArmor, SELinux) and full machine virtualization (KVM, Xen): the Sentry plus a filesystem proxy (the *Gofer*) compose a third architectural category that sits between the two. The documentation acknowledges that the tradeoff for this isolation is higher per-system-call overhead and reduced application compatibility, and notes that gVisor itself runs the Sentry inside a restricted seccomp container for defense-in-depth. gVisor ships an OCI-compatible runtime called **runsc** that integrates with Docker and Kubernetes.

Relevance to SafeExec. gVisor is one of the project's two named isolation back-ends, and this documentation is the primary source for its architecture, security model, and stated performance tradeoff. The "third path between syscall filtering and virtualization" framing is exactly the conceptual axis along which the project's evaluation compares its two back-ends.

Verification status (2026-05-13): Read https://gvisor.dev/docs/architecture_guide/security/ and https://gvisor.dev/docs/architecture_guide/performance/ before final submission, and consider adding the Security Model page as a separate citation.

[4] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI '20)*, Santa Clara, CA, USA, Feb. 2020, pp. 419–434. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/agache>

Firecracker is an open-source Virtual Machine Monitor (VMM) developed at AWS, specialized for serverless and container workloads, that uses Linux KVM to launch minimal "microVMs" with strong VM-level isolation while preserving the low startup latency and density traditionally associated with containers. The paper argues that the conventional tradeoff between strong isolation (full VMs) and minimal overhead (containers) is not acceptable for multi-tenant serverless platforms, and presents Firecracker as a counter-example: at the time of publication it backed AWS Lambda and Fargate at the

scale of millions of workloads and trillions of monthly requests. The design emphasizes a small VMM attack surface (written in Rust, minimal device model) as the primary security property.

Relevance to SafeExec. Firecracker is named in the project charter as the stretch-goal third isolation back-end (W10 only if ahead). Even if Firecracker is not implemented in the artifact, this paper establishes the strongest production-realistic point on the isolation-strength axis against which the project's hardened-Docker and gVisor back-ends are compared. It also frames the central tradeoff the project measures empirically.

Verification status (2026-05-13): Downloaded the paper from <https://www.usenix.org/system/files/nsdi20-paper-agache.pdf> and read the architecture section and the security discussion before final submission.

C. LLM-agent code-execution products and OSS projects

[5] E2B (e2b-dev), *E2B: Open-source, secure environment with real-world tools for enterprise-grade agents*, GitHub repository, Apache-2.0 License, accessed May 13, 2026. [Online]. Available: <https://github.com/e2b-dev/E2B>

E2B is a widely-used open-source platform ($\approx 10.9k$ GitHub stars as of May 2026) that provides on-demand secure sandboxes for executing AI-generated code on behalf of LLM-driven agents. According to E2B's own materials and corroborating community write-ups, each sandbox runs as a Firecracker microVM with its own isolated kernel, root filesystem, and network namespace, and is exposed to client agents through JavaScript and Python SDKs. The platform supports Python and JavaScript code execution, command APIs, background jobs, isolated filesystems, persistent volumes, and optional internet access — a feature set that defines the upper-bound expectation for "what a production agent sandbox provides."

Relevance to SafeExec. E2B is the most prominent open-source instance of the artifact category SafeExec is designed for, and is therefore the most important "what already exists" baseline against which SafeExec must position itself. SafeExec is *not* an attempted clone of E2B — its differentiator is a documented threat model and a reproducible adversarial benchmark rather than a SaaS-grade feature surface — and a well-written related-work section in the final report will need to make this distinction explicitly.

Verification status (2026-05-13): Located and read E2B's own architecture / security documentation at <https://e2b.dev/docs> to confirm the Firecracker claim and the threat-model statements.

D. Evaluation methodology for agentic / coding systems

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. 12th Int. Conf. Learning Representations (ICLR 2024)*, Vienna, Austria, May 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>

SWE-bench is a benchmark of 2,294 software-engineering tasks drawn from real GitHub issues and corresponding pull requests across 12 popular Python repositories; given a codebase and an issue description, the model under test must produce a patch that resolves the issue. The benchmark is

notable for two reasons relevant to this project. First, it pioneers a deterministic, *test-execution-based* grading methodology that does not require human judgement on outputs, which is the methodological pattern the project's functional and adversarial test suites adopt. Second, the SWE-bench evaluation harness itself uses Docker containers to execute each evaluation instance reproducibly — a direct precedent for the project's reproducibility expectations.

Relevance to SafeExec. SWE-bench is cited not because its task domain matches the project's (it does not), but because its evaluation-design philosophy — deterministic graders, containerized execution, public leaderboards, an explicit "Lite" / "Verified" tiering — is the methodological model the project's adversarial benchmark will follow.

Verification status (2026-05-13): Read the paper (arxiv.org/abs/2310.06770), and considered whether SWE-bench Verified (the OpenAI-curated subset) is the better citation for the reproducibility-methodology argument given that this annotation makes.

E. Adversarial / red-team corpora and case studies

[7] Red Hat Product Security, *runc - Malicious container escape - CVE-2019-5736*, Red Hat Customer Portal, Feb. 11, 2019 (updated Sept. 3, 2021). [Online]. Available: <https://access.redhat.com/security/vulnerabilities/runcescape>

CVE-2019-5736 is a now-classic container-escape vulnerability in **runc** (the OCI runtime underlying Docker, containerd, CRI-O, and others), in which a process running as UID 0 inside a default-configured container could overwrite the host's **runc** binary by exploiting the special behavior of the **/proc/self/exe** symlink and thereby gain root-level access on the host. The Red Hat advisory documents the attack mechanism, the patch (creating a sealed memory-backed copy of **runc** per launch), and pre-patch mitigations: running containers as non-root, mounting the host **runc** read-only, and enforcing an MAC system (the advisory specifically notes that SELinux in enforcing mode prevented exploitation). The vulnerability was credited to Adam Iwaniuk and Borys Popławski.

Relevance to SafeExec. CVE-2019-5736 is a canonical, well-documented example of a real-world isolation-failure attack class that a "default-Docker" sandbox configuration would have been vulnerable to. It is exactly the kind of incident from which the project's adversarial-test-suite taxonomy should be derived: the categories "attempted host binary write" and "attempted privilege escalation via **/proc/self/exe**" should appear in the adversarial suite specifically because real escapes have used these channels. More broadly, this entry seeds a "container escape CVE case studies" subsection that should grow to 3–5 entries by W11 (e.g., CVE-2022-0185 for the mount namespace; CVE-2024-21626 also in **runc**).

Verification status (2026-05-13): Read one detailed technical writeup (e.g., the AWS blog post linked from the search results, and the Frichetten PoC repository) to ground the adversarial-test-suite category derivation, and verified the CVE number against the NVD page (<https://nvd.nist.gov/vuln/detail/cve-2019-5736>).

Sources identified but not yet fetched

The following sources were identified during initial searching but not yet retrieved. Each is a credible candidate for promotion to a full entry as the bibliography grows toward the W11 ≥ 15 -source target. Recorded here so they are not forgotten.

- E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The True Cost of Containing: A gVisor Case Study," in *USENIX HotCloud '19* — academic performance analysis of gVisor, directly relevant to the project's performance comparison.
- Linux kernel documentation, *Seccomp BPF (SECure COMputing with filters)* — primary source for the seccomp filter that will be part of the hardened-Docker back-end.
- *seccomp(2)* — *Linux manual page* (man7.org) — primary syscall reference.
- A specific Anthropic or OpenAI publication on agent sandboxing (e.g., Code Interpreter documentation or a relevant model card) — to substantiate the problem-statement claim about closed-source agent execution environments.
- A more recent runc or container-runtime CVE (e.g., CVE-2024-21626 "Leaky Vessels") — to demonstrate that escapes from default container configurations continue to occur and that this is not a solved problem.
- A foundational paper or recent survey on prompt injection in agentic systems (e.g., Greshake et al., 2023, on indirect prompt injection) — to anchor the upstream-of-isolation threat model.
- The ML Reproducibility Checklist (Pineau et al.) — to anchor the reproducibility section of the project's evaluation plan.

How to grow this to 15 sources by W11

Aim for ≥ 3 entries per category. The current draft has 1 in A, 1 in B (gVisor) plus 1 in B (Firecracker), 1 in C (E2B), 1 in D (SWE-bench), 1 in E (CVE-2019-5736). To reach ≥ 3 per category, prioritize: in A, a prompt-injection paper plus a survey of agent-security risks; in B, the seccomp primary docs plus the gVisor case-study paper; in C, an Anthropic / OpenAI primary source for Code Interpreter or analysis tool, plus Modal Sandboxes or Daytona documentation; in D, the SWE-bench-Verified report plus one agent-eval benchmark like GAIA or AgentBench; in E, one more container CVE plus one academic adversarial-program corpus.
